Type System
type generic definition
<
type list
type list
type
>
type list
type
,
type
scoped identifier
type declaration
scoped identifier
Unclassified/Unspecified
literal value
number_literal
string_literal
char_literal
scoped identifier
::
identifier_path
identifier path part
identifier
identifier path part
identifier
::
capture block
{
capture block list
capture block list
capture block identifier list
capture block identifier list
identifier list of capture block
identifier list of capture block
&
scoped identifier
,
last identifier
&
scoped identifier
}
code block
{
code
}
identifier list
identifier list
identifier_list_elements
scoped identifier
,
CLua Programming Language
code
group of actions
group of actions
action
;
action
Expression List
Unary Expressions
reinterpret_cast
reinterpret_cast
type generic definition
(
simple expression
)
static_cast
static_cast
type generic definition
(
simple expression
)
unary minus
-
dot
.
unary hash
#
comma
,
unary ampersand
&
unary exclamation
!
size of
size_of
(
type
simple expression
)
align_of
align_of
(
type
simple expression
)
prefix minus
--
postfix minus
prefix plus
++
postfix plus
unary operator
complex unary operator
align_of
size_of
reinterpret_cast
cast
simple unary operator
unary minus
unary hash
unary ampersand
unary exclamation
unary compound operator
prefix minus
postfix minus
prefix plus
postfix plus
binary equal
compound operator
binary compound operator
N-ary Expression
export expression
export
export list
last export list element
export list
export list
export list element
export list element
identifier
->
scoped identifier
.
last export list element
identifier
->
scoped identifier
Binary Expressions
binary additive group
binary plus
+
binary minus
-
binary term group
binary asterisk
*
binary slash
/
binary equal
=
Meta Language
@
meta
Declaration
attribute list
attribute
attribute
attribute
any attribute
Statement
import
import
import specific
import all
{
identifier list
}
*
export
export
export specific
export all
{
identifier list
}
*
luau embedding block
luau
capture block
luau block
export expression
Statement
statement
statement block
{
expression
}
switch
switch
{
simple expression
}
case set
case set
case
full case
default case
case condition
case
literal value
scoped identifier
:
case
full case
case
literal value
scoped identifier
:
code block
default case
case
default
:
code block
if
if
statement block
code block
elseif block
elseif block
elseif
statement block
code block
else block
else
code block
for loop
for
(
expression
declaration
;
simple expression
;
expression
)
code block
foreach loop
foreach
(
variable declaration
simple expression
)
simple expression
)
code block
while loop
while
statement block
code block
do while loop
do
code block
while
statement block
Declaration
attribute list
declaration
function declaration
variable declaration
class declaration
enum declaration
namespace declaration
fun
scoped identifier
:
type
explicit variable declaration
variable mutability definition
scoped identifier
:
type
implicit type variable definition
variable mutability definition
scoped identifier
variable assignement
variable mutability definition
let
const
variable assignement
binary equal
simple expression
Expression
expression
simple expression
compound expression
additive expression
additive expression group
additive expression element
binary compound expression
unary compound expression
simple expression
binary compound operator
simple expression
unary compound operator
simple expression
unary compound operator
indexing expression
member access expression
member access atom
group expression
member access expression
scoped identifier
dot
group expression atom
atom
indexing expression atom
member access atom
identifier
simple expression
atom
literal value
identifier path
local expression
local expression
(
simple expression
)
indexing expression atom
[
simple expression
]
group expression atom
atom
simple expression
comma
call expression
call expression atom
call expression atom
indexing expression
call expression atom signature
(
group expression
)
primary expression
prefix expression
prefix expression
prefix expression atom
postfix expression
postfix expression atom
postfix expression atom
prefix expression atom
indexing expression
member access expression
call expression